

Molang Documentation - Introduction to Molang

08/13/2024

Molang is a simple expression-based language designed for fast, data-driven calculation of values at run-time, and with a direct connection to in-game values and systems. Its focus is to enable low-level systems like animation to support flexible data-driven behavior for both internal and external creators, while staying highly performant.

Molang uses the "min_engine_version" from the manifest.json of the resource or behavior pack that contains each Molang expression to determine which version of the rules to apply. This allows for changes to how Molang works without breaking existing content.

Molang Versioned Change versions apply to each expression separately, so it's possible to have different versions active if multiple packs are loaded.

This is a list of the Versioned Changes that have been added, along with the corresponding game version.

To know which Versioned Changes are in effect, look at the "min_engine_version" of the manifest.json of the resource or behavior pack that contains your Molang expression. Any Versioned Change with a version less than or equal to that version number will be in effect.

Versioned Change Versions

Pack min_engine_version	Description

1.17.0	Initial support for Versioned Changes added. (Not actually a Versioned Change)
1.17.30	Fixed query.item_remaining_use_duration conversion from ticks to seconds (multiplied by 20 instead of dividing). Also fixed normalization logic in that query to go from 1 to 0 instead of 0 to 1.
1.17.40	Added new error messages for invalid expressions which previously ran with unexpected results. For example "'text' + 1" will now cause a content error.
1.17.40	Added error detection for too many operators in parentheses or brackets, such as '1+(2 3)'. Also added more explicit error detection for when an unknown token is encountered.
1.18.10	Fixed conditional (ternary) operator associativity. Previously nested conditional expressions like $A ? B : C ? D : E$ would evaluate as $(A ? B : C) ? D : E$. Now they evaluate as $A ? B : (C ? D : E)$.
1.18.20	Fixed Logical AND to evaluate before Logical OR, and for comparison operators to evaluate before equality operators.
1.19.60	Fixed error where dividing by a dynamically determined negative variable resulted in a division by the absolute (positive) value of the number instead.
1.20.0	Fixed query.cape_flap_amount using the wrong player rotation (head rotation instead of body rotation).
1.20.10	Renamed block_property and has_block_property to block_state and has_block_state post this version.
1.20.40	Deprecated block_property and has_block_property
1.20.50	block_property is no longer supported, as that logic is done in the animation. Additionally, is_scenting is_rising and is_feelingHappy queries are no longer available; timer_flag_1, timer_flag_2, and timer_flag_3 can be used instead.
	Queries surface_particle_texture_size, surface_particle_texture_coordinate, and

1.20.70	surface_particle_texture_size now register leaf blocks as supporting for step particles
---------	---

The language structure is largely based on simple 'C' language family style syntax. An expression can be made of either one simple value or math calculation, or can be made of several sub-expressions where more complicated code is required.

In simple cases, the terminating ; is omitted and the expression result is returned. In complex cases, multiple sub-expressions are each terminated with a semicolon ;. Complex expressions evaluate to 0.0 unless there is a return statement, in which case the evaluated value of the return's sub-expression will be returned out of the current scope.

Note

All things in Molang are **case-insensitive**, with the exception of strings, which maintain the case provided.

All identifiers not in a scope listed below are reserved for future use.

Caution

Some operators are under Experimental Molang Features at the moment in the list below. We are hoping people will experiment with them and give us feedback, so we can move them into general usage.

Keyword	Description
1.23	Numerical constant value
! && < <= >= > == !=	Logical operators
* / + -	Basic math operators

<code>()</code>	Parentheses for expression term evaluation control
<code>{ }</code>	Braces for execution scope
<code>??</code>	Null coalescing operator, for handling missing variables or stale actor references
<code>geometry.texture_name</code>	A reference to a geometry named in the entity definition
<code>material.texture_name</code>	A reference to a material named in the entity definition
<code>texture.texture_name</code>	A reference to a texture named in the entity definition
<code>math.function_name</code>	Various math functions
<code>query.function_name</code>	Access to an entity's properties
<code>temp.variable_name</code>	Read/write temporary storage
<code>texture.texture_name</code>	A reference to a texture named in the entity definition
<code>variable.variable_name</code>	Read/write storage on an actor
<code>temp.variable_name</code>	Read/write temporary storage
<code>context.variable_name</code>	Read-only storage provided by the game in certain scenarios
<code><test> ? <if true> : <if false></code>	Ternary conditional operator. NOTE: Nested ternary expressions without parentheses were incorrectly parsed before a Versioned Change was made to fix it (see 'Versioned Changes' below).
<code><test> ? <if true></code>	Binary conditional operator
<code>this</code>	The current value that this expression will ultimately write to (context specific)
<code>return</code>	For complex expressions, this evaluates the following statement and stops execution of the expression, returns the value computed
<code>-></code>	Arrow operator, for accessing data from a different entity
<code>context.variable_name</code>	Read-only storage provided by the game in certain scenarios
<code>loop</code>	For repeating one or more commands 'n' times

<code>for_each</code>	For iterating over an array of entities
<code>break</code>	For early exiting a loop/for_each scope
<code>continue</code>	For skipping the rest of the set of statements of a loop/for_each iteration and moving to the next iteration
<code>[]</code>	Brackets for array access

There are three variable lifetimes a variable may belong to: Temporary, Entity, and Context:

- **Temporary variables** (eg: `temp.moo = 1;`) are read/write and valid for the scope they are defined in, as per C rules. For performance reasons their lifetime is global to the current expression execution and may return a valid value outside of the outermost scope they are defined in for a expression. Be careful in complex expressions. We will be adding content errors for invalid accesses as soon as possible.
- **Entity variables** (eg: `variable.moo = 1;`) are read/write and store their value on the entity for the lifetime of that entity. Note that these are currently not saved, so quitting and reloading the world will re-initialize these. In the same way, if the entity is despawned, any variables on the entity will be lost.
- **Context variables** (eg: `context.moo`) are read-only and valid for the expression they are run on. The game defines these, and details on what variables are in each will be available in the documentation of the area where that Molang expression exists (such as behaviors defining what context variables they expose).

In general, variables of a mob are considered private to that mob and cannot be accessed by another. To expose read-only access of a variable to other mobs, you need to set the 'public' setting on that variable in the owning entity's resource definition. It is also recommended to default-initialize the

variable.

```
{
  "format_version": "1.10.0",
  "minecraft:client_entity": {
    "description": {
      "scripts": {
        "variables": {
          "variable.oink": "public"
        },
        "initialize": [
          "variable.oink = 0;"
        ],
      },
    }
  }
}
```

-
- All numerical values are floats.
 - Boolean values such as actor flags are converted and stored as a float value of either 0.0 or 1.0 for values of false or true respectively.
 - For boolean tests, a float value equivalent to 0.0 is false, and anything not equal to 0.0 is true.
 - For array indices, floats are C-style-cast to ints, and clamped at zero for negative values or wrapped by the array size for large values.
 - Errors (such as divide by zero, missing variables, null references, etc) generally return a value of 0.0.

Molang supports the following value types as well:

Geometry

Texture

Material

Actor Reference

Actor Reference Array

String

Struct

Tip

See the Structs section below to learn more on how Molang handles them.

Query functions (eg: `query.is_baby` or `query.is_item_equipped('main_hand')`) allow expressions to read game data. If a query function takes no arguments, do not use parentheses. Otherwise, use parentheses and separate arguments with commas. For a full list of query functions, click on the hyperlink to view [Query Functions](#).

To reduce typing burden and increase clarity when reading and writing Molang, the following keyword aliases can make life a bit easier.

Note

Note that left and right sides function identically in the Alias Map.

Full Name	Aliased Name
<code>context.moo</code>	<code>c.moo</code>
<code>query.moo</code>	<code>q.moo</code>
<code>temp.moo</code>	<code>t.moo</code>
<code>variable.moo</code>	<code>v.moo</code>

The following example shows how using aliases will keep the code short while functioning the same way.

```
math.cos(query.anim_time * 38) * variable.rotation_scale + variable.x * variable
```

```
math.cos(q.anim_time * 38) * v.rotation_scale + v.x * v.x * q.life_time
```

Molang will also allow you to use either syntax and intermix as desired as shown in this last example below.

```
math.cos(q.anim_time * 38) * variable.rotation_scale + v.x * variable.x * query.
```

One difference between Molang and the C style syntax is that structures of data are **implicitly** defined by usage. Their purpose is to more efficiently pass data around, such as passing `v.location` rather than `v.x`, `v.y`, and `v.z`. An example of this is shown below:

```
v.location.x = 1;  
v.location.y = 2;  
v.location.z = 3;  
v.another_mobs_location = v.another_mob_set_elsewhere->v.location;
```

For some more usage examples, each of the following expressions return 1.23

```
v.cowcow.friend = v.pigpig; v.pigpig->v.test.a.b.c = 1.23; return v.cowcow.frien
```

```
v.cowcow.friend = v.pigpig; v.pigpig->v.test.a.b.c = 1.23; v.moo = v.cowcow.frie
```

```
v.cowcow.friend = v.pigpig; v.pigpig->v.test.a.b.c = 1.23; v.moo = v.cowcow.frie
```

```
v.cowcow.friend = v.pigpig; v.pigpig->v.test.a.b.c = 1.23; v.moo = v.cowcow.frie
```

```
v.cowcow.friend = v.pigpig; v.pigpig->v.test.a.b.c = 1.23; v.moo = v.cowcow.frie
```

Note that structures can be arbitrarily deep in their nesting/recursiveness. With that being said, it is recommended that you **do not** copy full structures inside other structures to avoid exploding memory, and not making structures too deep as there is a slight performance cost for each layer.

Strings in Molang are surrounded by single quotes, shown here as 'minecraft:pig' or 'hello world!'. An empty string is defined as two consecutive single quotes shown here as ''.

String operations only support == and != at this time.

Note

Strings don't support the ' character as there is no support for escape characters at this time.

Listed below are the mathematical functions available for use in Molang.

Function	Description
math.abs(value)	Absolute value of value
math.acos(value)	arccos of value

<code>math.asin(value)</code>	arcsin of value
<code>math.atan(value)</code>	arctan of value
<code>math.atan2(y, x)</code>	arctan of y/x. NOTE: the order of arguments!
<code>math.ceil(value)</code>	Round value up to nearest integral number
<code>math.clamp(value, min, max)</code>	Clamp value to between min and max inclusive
<code>math.cos(value)</code>	Cosine (in degrees) of value
<code>math.die_roll(num, low, high)</code>	returns the sum of 'num' random numbers, each with a value from low to high. Note: the generated random numbers are not integers like normal dice. For that, use <code>math.die_roll_integer</code> .
<code>math.die_roll_integer(num, low, high)</code>	returns the sum of 'num' random integer numbers, each with a value from low to high. Note: the generated random numbers are integers like normal dice.
<code>math.exp(value)</code>	Calculates e to the value 'nth' power
<code>math.floor(value)</code>	Round value down to nearest integral number
<code>math.hermite_blend(value)</code>	Useful for simple smooth curve interpolation using one of the Hermite Basis functions: $3t^2 - 2t^3$. Note that while any valid float is a valid input, this function works best in the range [0,1].
<code>math.lerp(start, end, 0_to_1)</code>	Lerp from start to end via 0_to_1
<code>math.lerprootate(start, end, 0_to_1)</code>	Lerp the shortest direction around a circle from start degrees to end degrees via 0_to_1
<code>math.ln(value)</code>	Natural logarithm of value
<code>math.max(A, B)</code>	Return highest value of A or B
<code>math.min(A, B)</code>	Return lowest value of A or B
<code>math.min_angle(value)</code>	Minimize angle magnitude (in degrees) into the range [-180, 180)
<code>math.mod(value, denominator)</code>	Return the remainder of value / denominator
<code>math.pi</code>	Returns the float representation of the constant pi.

<code>math.pow(base, exponent)</code>	Elevates base to the exponent'th power
<code>math.random(low, high)</code>	Random value between low and high inclusive
<code>math.random_integer(low, high)</code>	Random integer value between low and high inclusive
<code>math.round(value)</code>	Round value to nearest integral number
<code>math.sin(value)</code>	Sine (in degrees) of value
<code>math.sqrt(value)</code>	Square root of value
<code>math.trunc(value)</code>	Round value towards zero

Brace Scope Delimiters { }

One can group a series of statements into a single group by wrapping them in { and } symbols. This is used primarily in loops and conditional statements:

```
(v.moo > 0) ? {  
    v.x = math.sin(q.life_time * 45);  
    v.x = v.x * v.x + 17.3;  
    t.sin_x = math.sin(v.x);  
    v.x = t.sin_x * t.sin_x + v.x * v.x;  
    v.x = math.sqrt(v.x) * v.x * math.pi;  
}
```

The conditional '?' operator allows for two convenient ways to implement simple branching logic. The first way is to use '?' by itself to conditionally execute part of an expression, for example `A ? B`. The part after the '?' is only run if the part before it evaluates to a true boolean. The second way is to use '?' with a ':' as a 'conditional ternary', for example `A ? B : C`. If the part before the '?' is evaluated as true, the part before the ':' is returned.

Otherwise the part after is returned.

Note

Nested ternary expressions without parentheses were incorrectly parsed before a Versioned Change was made to fix it (see 'Versioned Changes' below).

```
v.should_reset_a ? { v.a = 0; }  
v.larger_value = (v.a > v.b) ? v.a : v.b;
```

Sometimes you may want to execute an expression multiple times. Rather than copy-pasting it a bunch, you can use `loop(<count>, <expression>);`. We have placed some arbitrary restrictions on these for safety for now. The maximum loop counter is 1024.

Caution

Also, note that while you can nest loops inside loops pretty much as deep as you want, be careful you don't make a loop so long it will hang your game.

The example below showcases how a Fibonacci Calculator can be written in Molang.

```
v.x = 1;  
v.y = 1;  
loop(10, {  
    t.x = v.x + v.y;  
    v.x = v.y;  
    v.y = t.x;  
});
```

This will exit out of a `loop` or `for_each` early.

```
v.x = 1;  
v.y = 1;  
loop(10, {t.x = v.x + v.y; v.x = v.y; v.y = t.x; (v.y > 20) ? break;});
```

This will immediately exit the inner-most active loop, as per C-style language rules. If you have:

```
v.x = 0;  
loop(10, {loop(10, {v.x = v.x + 1; (v.x > 5) ? break;});});
```

The `break` statement will terminate the inner loop when `v.x > 5`, and continue processing the outer loop's expression. Note that as `v.x` is not reset between the outer loops, the second time into the inner loop this will add one more to `v.x` and then exit the inner loop again, resulting in a final value of `v.x` of $6 + 1 + 1 + 1 + \dots + 1 = 15$.)

`continue` functions as per C-style language rules. Currently only supported in `loop` and `for_each`, this will skip to the next iteration of the current loop. See `break` above for more details on inner/outer loops.

The following example will result in `v.x` becoming 6.0, as the increment will be skipped once it reaches that value. Note that it is better to break out of the loop in this contrived example, as it would be more performant than continuing to perform all 10 iterations.

```
v.x = 0;
```

```
loop(10, {  
    (v.x > 5) ? continue;  
    v.x = v.x + 1;  
});
```

Null Coalescing Operator ??

Similar to how the null-coalescing operator works in C#, you can now reference a variable that may or may not exist without seeing a content error. If it doesn't, you can now provide a default value to use. Previously, if a variable didn't exist you would get a content error. This was to make sure variables were always initialized correctly to avoid uninitialized variable bugs.

Unfortunately this then required initialized scripts, or in some cases some complex work-arounds to make sure variables were initialized. Now, if you know a variable won't be initialized in the first run of a script, you can use the following:

```
variable.x = (variable.x ?? 1.2) + 0.3;
```

This will use the value of `variable.x` if it is valid, or else 1.2 if `variable.x`:

- has not yet been initialized
- is a reference to a deleted entity
- is an invalid reference
- holds an error

Note that the `??` operator will work with `variable.s`, `temp.s`, and `context.s` that hold numbers or entity references, but not resources such as materials, textures, or geometries (as those must exist and be valid else it's a content

error). If the first argument would result in something that can't be resolved, it will return the second argument.

Important

Reminder: the standing rule of thumb in Molang is that if something would error or be a bad value, it is converted to 0.0 (and generally throw a content error on screen in non-publish builds. Note that content errors may prevent uploading content to the marketplace, so please ensure expressions aren't going to do bad things such as dividing by zero).

Simple vs Complex Expressions

A simple expression is a single statement, the value of which is returned to the system that evaluated the expression. This is showcased in the example below.

```
math.sin(query.anim_time * 1.23)
```

A complex expression is one with multiple statements, each ending in a ';'. Each statement is evaluated in order. In the current implementation, the last statement requires the use of the return keyword and defines the resulting value of the expression as shown here:

```
temp.moo = math.sin(query.anim_time * 1.23);  
temp.baa = math.cos(query.life_time + 2.0);  
return temp.moo * temp.moo + temp.baa;
```

Note that in a simple expression, ; is not allowed, whereas in a complex

expression, each statement requires a ; including the last. Also, note that if you don't return a value from a complex expression, the expression will evaluate to 0.0.